

Anomalous Sound Detection

Abdessamed Qchohi
EURECOM

Abdessamed.Qchohi@eurecom.fr

Francesco Giannuzzo
EURECOM

Francesco.Giannuzzo@eurecom.fr

Alessio Giuffrida
EURECOM

Alessio.Giuffrida@eurecom.fr

Matteo Ghia
EURECOM

Matteo.Ghia@eurecom.fr

Abstract—This project addresses the problem of anomalous sound detection in the context of the DCASE 2020 Challenge on the detection and classification of acoustic scenes and events [1]. The objective is to detect unknown anomalous sounds given a dataset that contains only normal sound samples. We experimented with various approaches, firstly employing traditional machine learning models like One-Class SVM, Gaussian Mixture Model and Isolation Forest, and then we used a Convolutional Autoencoder, which is a deep learning model.

I. INTRODUCTION

The objective of this work is to build a machine learning model that is able to identify whether the sound emitted by an industrial machine is normal or anomalous. This challenge is closely related to the industrial problem of automatically detecting mechanical failure to achieve factory automation. The machines are subdivided according to their type and each machine is characterized by a unique ID. The original dataset contains sound samples for six types of toy/real machines, but for the purpose of this project we considered only one machine type, i.e. Slide rail. Moreover, because collecting patterns of anomalous sounds is practically impossible, the training dataset contains only normal samples, whereas the test dataset contains also anomalies.

II. DATASET ANALYSIS

A. Dataset description

The dataset provided is derived from the MIMII dataset [2], which focuses on slide rail machines. Each recording is a 10-second, single-channel audio file that captures operating sounds and ambient factory noise. The dataset includes normal and anomalous sounds, with anomalous sounds simulated by deliberately damaging the machines. The sampling rate of all signals has been downsampled to 16 kHz. The dataset consists of a development dataset, an additional training dataset and an evaluation dataset:

- the development dataset is made up of two different parts: a training set, consisting of 2370 normal sounds of machines with 3 different IDs, and a test set, which contains 1101 sounds (300 normal, 801 anomalous). Fig. 1 and Fig. 2 shows the distribution of the recordings according to the machine ID in the training and test sets.

- the evaluation dataset contains recordings that have no label and are characterized by different machine IDs from the ones in the development dataset.
- the additional training dataset includes normal sounds characterized by the same machine IDs present in the evaluation dataset.

We noticed that the test set in the development dataset contained a good amount of anomalous sounds. Moreover, we did not have enough information about the type of sounds that characterize an anomaly: we therefore concluded that trying to artificially add noise could worsen the quality of our data. For these reasons we decided not to perform data augmentation.

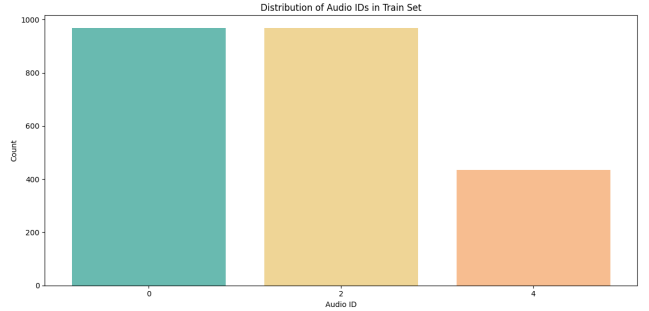


Fig. 1. Distribution of the audios in the training set

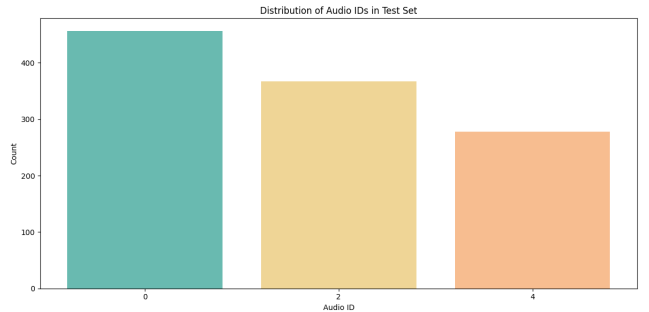


Fig. 2. Distribution of the audios in the test set

B. Waveform analysis

The first step of our analysis was to visualize the audio signals to have a better understanding of their characteristics.

Firstly we plotted the waveforms of normal and anomalous signals: Fig. 3 shows that anomalous signals demonstrate an irregular behaviour, with sudden peaks in amplitudes.

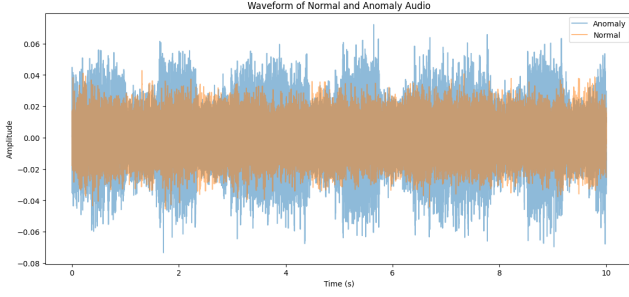


Fig. 3. Comparison of spectrum of normal and anomalous sounds

C. Fourier Transform

Since we are dealing with audio signals, we decided to compute their Fourier transforms to understand if it was possible to identify anomalous sounds based on some patterns in the frequency domain. From Fig. 4 we can conclude that the spectrum of anomalies presents some differences, in particular regarding the number of peaks and the mean magnitude, which on average are both higher than normal sounds. We therefore concluded that these could be useful features for anomaly detection.

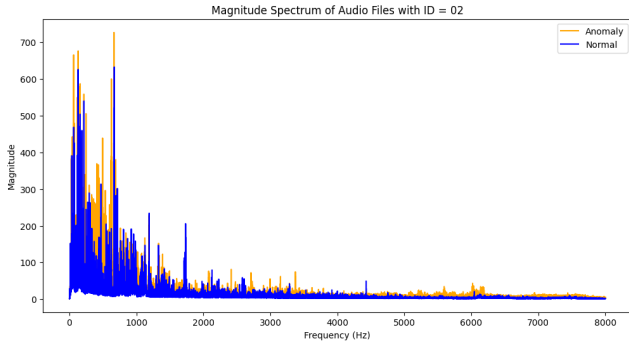


Fig. 4. Comparison of spectrum of normal and anomalous sounds

D. Spectrograms

To further improve our analysis, we computed the Mel spectrograms of the signals, obtaining a representation of the frequency spectrum as a function of time. Fig. 5 compares the Mel spectrogram of a normal and an anomalous sound: in this plot the color intensity represents the amplitude of frequency components at a particular time. The first spectrogram is characterized by regular frequency patterns with an uniform distribution of colors, whereas the second one has an irregular color pattern, reflecting its anomalous nature. We also plotted the 3D spectrograms (Fig. 6), which show the same behaviour.

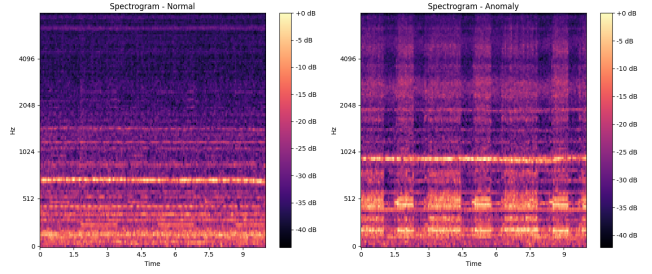


Fig. 5. Mel spectrogram of normal and anomalous sound

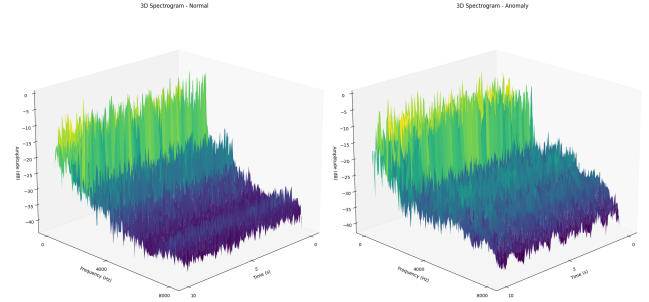


Fig. 6. 3D spectrogram of normal and anomalous sound

III. DATA PREPROCESSING AND PREPARATION

A. Feature extraction

The data preprocessing steps used differ between the traditional machine learning models and the deep learning model: for the first ones we extracted some features which we regarded as the most important to characterize the audio signals. These features include:

- peak-to-peak amplitude
- root mean square (RMS)
- number of peaks
- spectral entropy
- zero crossing rate
- signal to noise ratio (SNR)

Moreover, we computed for each audio signal the amplitude, the FFT magnitude and the Mel-Frequency Cepstral Coefficients (MFCC) magnitude. Then we also computed the mean values of these quantities separately for each machine ID and we used as features the euclidean distances between the signal's values for each quantity and the mean values corresponding to the signal's machine ID. This was done because signals coming from different machines exhibit different characteristics. At the end we obtained 13 features, but we wanted to check how they were correlated, so we plotted the correlation matrix of the transformed dataset as a heatmap, shown in Fig. 7. Since there is some significant correlation between the features, we decided it would be appropriate to employ dimensionality reduction techniques, in particular Principal Component Analysis (PCA). The number of principal components kept after applying PCA is a hyperparameter

to tune, therefore each model may use a different number of principal components.

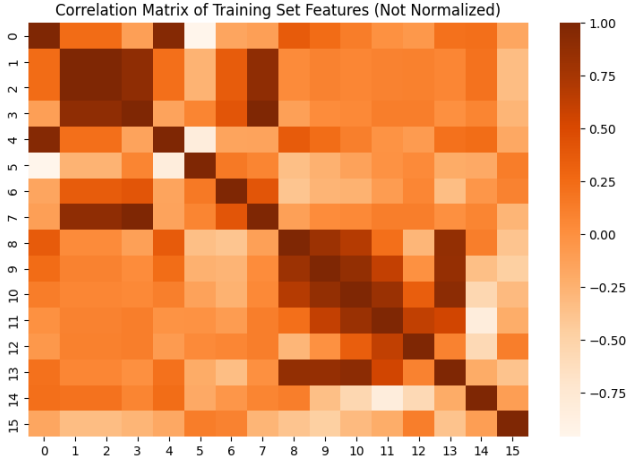


Fig. 7. Correlation matrix of the transformed dataset

For the Convolutional Autoencoder, instead of using these features, we converted all the audio signals to Mel spectrograms and used them directly as inputs. In both cases (traditional models and Autoencoder), the data has been normalized in order to have zero mean and unitary variance. An important observation to make is that we decided not to include the machine ID as a feature, as it would have as a consequence data leakage. Indeed, we wanted our models to be able to detect anomalies regardless of the machine, in order to achieve good generalization capabilities.

B. Dataset Splitting

Our work focused primarily on the development dataset. We used two different approaches for splitting the data:

- for the traditional machine learning models, we used the entire training set to train the models and we performed a 60/40 split on the test set: this allowed us to obtain a validation set, used for model selection and tuning, and a test set, used to assess the generalization capabilities of the models.
- for the Convolutional Autoencoder we performed a 80/20 split on the training set to obtain a validation set, used for tuning and to check for overfitting, while the entire test set was used for evaluation.

After having found the models' best hyperparameters, we retrained all models on the additional training dataset before evaluating them on the test set.

IV. MODEL SELECTION

To tackle the task at hand we experimented with different machine learning models:

- **One-Class SVM:** an unsupervised anomaly detection algorithm, which learns a decision function that captures the region in the feature space where the training data lies. At prediction time, it determines whether a new

observation falls within this learned region or outside it, as an anomaly.

- **Isolation Forest:** an unsupervised anomaly detection algorithm that isolates data points by randomly selecting a feature and then randomly selecting a split value between the maximum and minimum values of that feature. Anomalies, which are more susceptible to isolation, are isolated in fewer splits than normal points.
- **Gaussian Mixture Model (GMM):** a probabilistic model that assumes data is generated from a mixture of several Gaussian distributions with unknown parameters. Anomalies are then identified as data points that have a low probability of belonging to this model.
- **Convolutional Autoencoder:** a neural network that learns to compress and then reconstruct its input data. It has an encoder that reduces the input to a smaller representation and a decoder that tries to rebuild the original input from this reduced form. The difference between the original and reconstructed data, called reconstruction error, is high if the input sound is different from the learned normal sounds, potentially signaling an anomaly [3].

In order to evaluate the model we used as metric the Area Under the ROC (Receiver Operating Characteristic) Curve score, which gives a measure for how well the model can separate the two classes, regardless of the threshold chosen to distinguish between normal and anomalous sounds. For the traditional machine learning models we used the AUC score as a metric during both validation and testing, while for the Autoencoder we used the mean squared error (MSE), which is the reconstruction error, during the training and for hyperparameters' tuning, and the AUC score to evaluate the performance on the test set. For all the experiments we set the seed to 42 for the reproducibility of results and then we tested the models with 10 different seeds chosen at random to evaluate the variance of the results.

A. Isolation Forest

We used the Isolation Forest algorithm, training on the entire training set from the development dataset and performing hyperparameters' tuning with grid search on the hyperparameters in Table I, obtaining the following best configuration: **n_estimators: 500, max_samples: 0.5, max_features: 1.0, bootstrap: True, contamination: 0.1, number_pc: 4**, and metric's value on the validation set: **AUC score: 0.851**.

TABLE I
HYPERPARAMETERS ISOLATION FOREST

Model	Hyperparameter	Values
Isolation Forest	n_estimators	{200, 300, 500}
	max_samples	{'auto', 0.5, 0.7}
	max_features	{0.5, 1.0}
	bootstrap	{True, False}
	contamination	{0.1, 0.01}
	number_pc	{1, 2, 3, 4, 13}

We retrained the model with the best hyperparameters on the additional training dataset and we evaluated the performance

on the test set, obtaining the following result: **AUC score: 0.830**. Fig. 8 shows the ROC curve obtained. We also retrained the model with 10 different seeds and we obtained the following results: **Mean AUC: 0.823, variance: $2.6 \cdot 10^{-4}$** . We can therefore conclude that the model has good generalization capability.

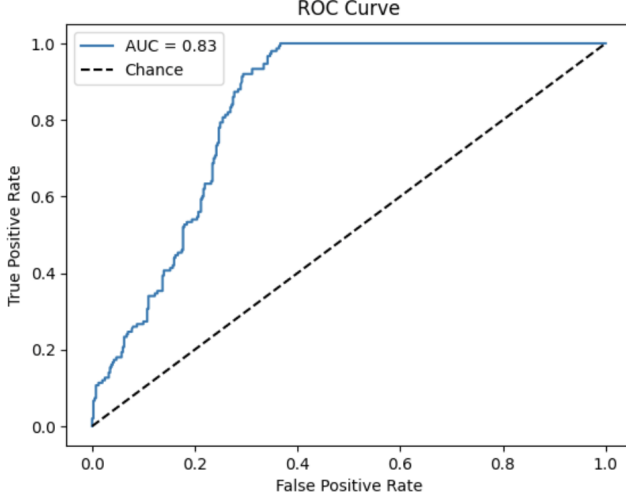


Fig. 8. Isolation Forest ROC curve

B. One-Class SVM

We implemented a One-Class SVM, training on the entire training set from the development dataset and performing hyperparameters' tuning with grid search on the hyperparameters in Table II, obtaining the following best configuration: **kernel: 'rbf', gamma: 0.1, nu: 0.3, number_pc: 13**, and metric's value on the validation set: **AUC score: 0.877**.

TABLE II
HYPERPARAMETERS ONE-CLASS SVM

Model	Hyperparameter	Values
One-Class SVM	kernel	{ 'rbf', 'poly', 'sigmoid' }
	gamma	{ 0.01, 0.1, 'scale', 'auto' }
	nu	{ 0.1, 0.3, 0.5 }
	number_pc	{ 1, 2, 3, 4, 13 }

We retrained the model with the best hyperparameters on the additional training dataset and we evaluated the performance on the test set, obtaining the following result: **AUC score: 0.852**. Fig. 9 shows the ROC curve obtained. We also retrained the model with 10 different seeds and we obtained the following results: **Mean AUC: 0.855, variance: $1.1 \cdot 10^{-4}$** . This model performs better than the One-Class SVM, but its results can be further improved on.

C. Gaussian Mixture Model

We implemented a Gaussian Mixture model, training on the entire training set from the development dataset and performing hyperparameters' tuning with grid search on the hyperparameters in Table III, obtaining the following best configuration: **covariance_type: 'full', max_iter: 3000, init_params:**

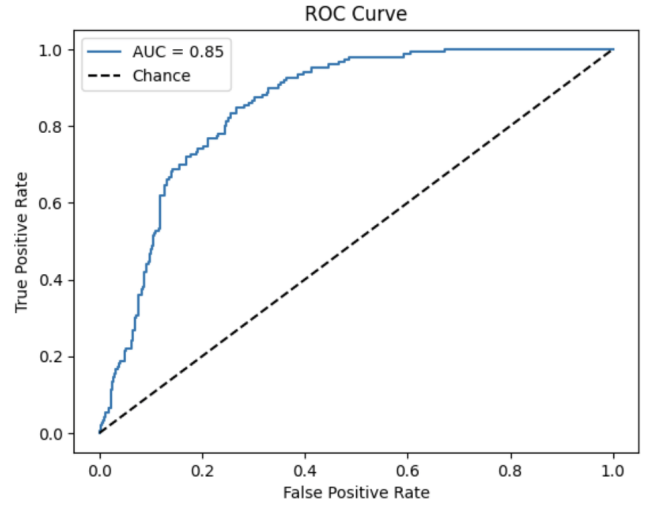


Fig. 9. One-Class SVM ROC curve

'kmeans', reg_covar: 1e-5, number_pc: 1, and metric's value on the validation set: **AUC score: 0.904**.

TABLE III
HYPERPARAMETERS GAUSSIAN MIXTURE MODEL

Model	Hyperparameter	Values
Gaussian Mixture Model	covariance_type	{ 'full', 'diag', 'spherical' }
	max_iter	{ 750, 1500, 3000 }
	init_params	{ 'kmeans', 'random' }
	reg_covar	{ 1e-6, 1e-5, 1e-4 }
	number_pc	{ 1, 2, 3, 4, 13 }

We retrained the model with the best hyperparameters on the additional training dataset and we evaluated the performance on the test set, obtaining the following result: **AUC score: 0.914**. Fig. 10 shows the ROC curve obtained. We also retrained the model with 10 different seeds and we obtained the following results: **Mean AUC: 0.906, variance: $1.0 \cdot 10^{-4}$** . The results obtained are really good, highlighting the good capabilities of this model on this task.

D. Convolutional Autoencoder

We implemented an Autoencoder architecture made up of 4 convolutional layers, with batch normalization and ReLU activation function. The neural network has been trained for 50 epochs with a batch size of 16, performing hyperparameters' tuning with grid search on the hyperparameters in Table IV. The hyperparameters chosen are used in the Adam optimizer and the StepLR scheduler employed during the training process. At the end of the tuning we obtained the following best configuration: **gamma: 0.5, step_size: 10, learning_rate: 1e-3**, and metric's value on the validation set: **MSE: 0.232**. Fig. 11 shows the trend of the reconstruction error on the training and validation set.

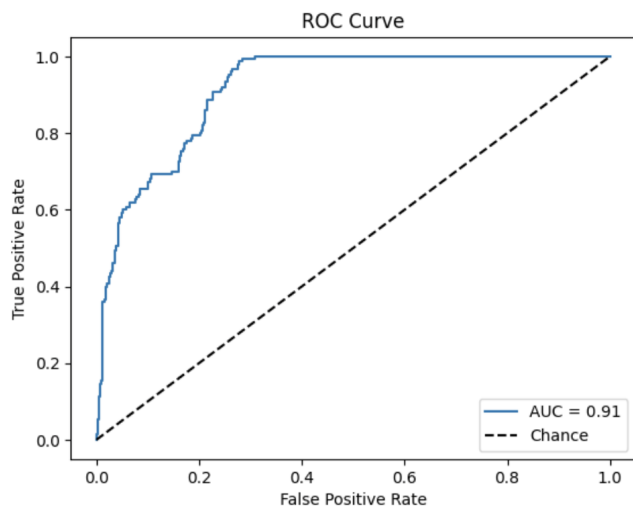


Fig. 10. Gaussian Mixture Model ROC curve

TABLE IV
HYPERPARAMETERS AUTOENCODER

Model	Hyperparameter	Values
Autoencoder	gamma	{0.01, 0.1, 0.5}
	step_size	{3, 5, 7, 10}
	learning_rate	{0.001, 0.01}

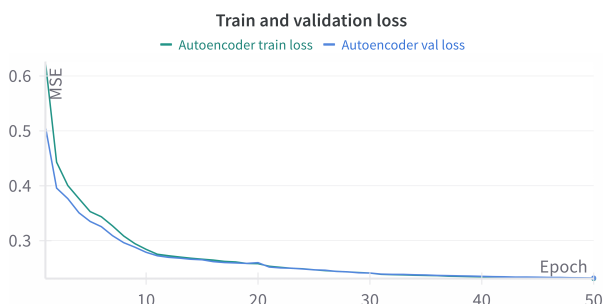


Fig. 11. Train and validation loss of the Autoencoder

We retrained the model with the best hyperparameters on the additional training dataset and we evaluated the performance on the test set, obtaining the following result: **AUC score: 0.782**. The most likely reason why the Autoencoder performs worse than the other models is that the number of training data is not large enough to fully exploit the computational capabilities of this architecture.

V. CONCLUSIONS AND FUTURE WORK

In this project we faced the problem of anomaly detection on a dataset consisting of audio signals coming from industrial machines. We analysed various characteristics of the signals, both in the time and in the frequency domain, and we computed the Mel spectrograms to obtain additional information. We experimented with both traditional machine learning models and deep neural networks, obtaining satisfactory results

with the traditional machine learning models, whereas the Autoencoder performed worse because of the lack of a large amount of training data. All the results are summarised in Table V.

TABLE V
RESULTS

Model	Test AUC score
Isolation Forest	0.830
One-Class SVM	0.852
Gaussian Mixture Model	0.914
Convolutional Autoencoder	0.782

We also tried to use a PANN (Pretrained Audio Neural Network) [4] to assign labels to the unlabeled evaluation dataset by extracting embeddings from the audio files and passing these embeddings as input to a Local Outlier Factor (LOF) algorithm [5]. However, we noticed that the algorithm was not able to identify anomalies, as almost all the sounds were predicted as normal: we assumed that it was unlikely that the evaluation dataset contained mostly normal sounds, so we decided to discard this idea. As possible future improvements, it would be possible to find some good strategies to assign labels to the unlabeled test, as well as finding a way to implement effective data augmentation.

REFERENCES

- [1] DCASE 2020 Challenge. <https://dcase.community/challenge2020/index>
- [2] Harsh Purohit, Ryo Tanabe, Takeshi Ichige, Takashi Endo, Yuki Nikaido, Kaori Suefusa, and Yohei Kawaguchi. "MIMII Dataset: Sound Dataset for Malfunctioning Industrial Machine Investigation and Inspection". In Proceedings of the Detection and Classification of Acoustic Scenes and Events 2019 Workshop (DCASE2019), November 2019.
- [3] Zhaomin Chen, Chai Kiat Yeo, Bu Sung Lee, and Chiew Tong Lau. "Autoencoder-based network anomaly detection". In 2018 Wireless telecommunications symposium (WTS), IEEE, 2018.
- [4] Qiuqiang Kong, Yin Cao, Turab Iqbal, Yuxuan Wang, Wenwu Wang, and Mark D. Plumbley. "Panns: Large-scale pretrained audio neural networks for audio pattern recognition." IEEE/ACM Transactions on Audio, Speech, and Language Processing 28, 2020.
- [5] LOF algorithm. https://github.com/scikit-learn/scikit-learn/blob/main/sklearn/neighbors/_lof.py